

FastMatMul: A Code-Based Matrix-Multiplication Checking Layer for Verifiable Computation

Anonymous Submission

Abstract—Verifiable computation systems frequently need to prove large matrix multiplication statements, yet this primitive remains a severe bottleneck for SNARK-based approaches: naïve arithmetization of a $k \times k$ product incurs $\mathcal{O}(k^3)$ constraints, and even the classical Freivalds reduction yields $\mathcal{O}(k^2)$.

We present **FastMatMul**, a matrix-multiplication checking layer for verifiable computation that reduces the matrix-checking SNARK circuit size to $\mathcal{O}(tk + t \log n + E_{cc}(k))$, where t is a small security-parameter-dependent constant and $E_{cc}(k)$ is the cost of the selected code-commitment backend. With a linear-time code-commitment backend, this becomes $\mathcal{O}(tk)$. The protocol combines a structured variant of Freivalds’ randomized check with *proximity testing over linear error-correcting codes*: matrices are encoded row-wise and committed via Merkle trees; the verifier detects any inconsistency by querying t random codeword positions and checking lightweight algebraic fold relations. The expensive $\mathcal{O}(k^3)$ matrix arithmetic is performed entirely outside the SNARK. We prove that this checking layer achieves perfect completeness and computational soundness, relative to certified encoded input commitments and the code-commitment backend, with error $\epsilon_{\text{SNARK}} + \epsilon_{cc} + \epsilon_{\text{bind}} + (k-1)/|\mathbb{F}| + 4(1-\delta)^t$, where δ is the relative distance of the underlying code. The Fiat–Shamir non-interactive variant incurs the standard additional random-oracle loss $\epsilon_{\text{FS}}(q_H)$. The result is therefore modular: an end-to-end system must instantiate and certify the input-commitment and code-commitment backend assumed by the theorem.

We implement the multiplication-checking layer of **FastMatMul** in Rust and evaluate it against a direct Freivalds SNARK baseline under this conditional linear-time backend model. In the current QA-NIZK implementation, experiments show a $7.5\times$ constraint reduction at $k = 1,024$ and a $30.3\times$ reduction at $k = 4,096$ for this layer. At $k = 4,096$, **FastMatMul_{QA}** achieves a $7.18\times$ proving-time speedup and produces a 104 KB proof; it continues to $k = 8,192$ with a 126 KB proof. A GPT-2 workload case study with 36 matrix-multiplication claims shows a $2.30\times$ constraint reduction and a $1.77\times$ proving-time speedup.

Index Terms—verifiable computation, matrix multiplication, SNARKs, code commitments, proximity testing

1. Introduction

Large matrix multiplication is a recurring bottleneck in verifiable computation. A prover may wish to convince a verifier that committed matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ satisfy $\mathbf{AB} = \mathbf{C}$ without forcing the verifier, or an underlying SNARK circuit, to recompute the full product. This problem appears across outsourced linear algebra, proof-carrying data pipelines, and verifiable inference for modern neural networks, where projection and attention layers routinely contain matrix products with dimensions $k \geq 1,000$. Naïve SNARK arithmetization of a single $k \times k$ matrix multiplication generates $\mathcal{O}(k^3)$ arithmetic constraints, a cost that is impractical even for moderate k .

The $\mathcal{O}(k^2)$ barrier. Freivalds’ classical randomized check [1] reduces the problem from $\mathcal{O}(k^3)$ to $\mathcal{O}(k^2)$: given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and a random vector $\mathbf{r}$, one verifies $\mathbf{rAB} = \mathbf{rC}$ instead of recomputing the full product. Several recent systems, including verifiable ML pipelines such as Mystique [2], zkML [3], and the Freivalds-based baseline in [4], adopt this strategy, encoding the vector-matrix products directly as SNARK constraints. The resulting $\mathcal{O}(k^2)$ circuit is an improvement, yet it remains expensive: in our optimized baseline, Freivalds already requires 50.5 million R1CS constraints and 405.04 s of proving time at $k = 4,096$. Thus the gap between the $\mathcal{O}(k^2)$ SNARK cost and practical large-matrix verification remains wide.

Our insight: from computing to checking. We observe that the $\mathcal{O}(k^2)$ barrier is *not* inherent to the verification problem—it arises because existing approaches *compute* the vector-matrix products inside the SNARK circuit. Our key idea is to shift the paradigm from “computing in the circuit” to “*checking* in the circuit”: the prover performs the expensive matrix arithmetic *outside* the SNARK and commits to all intermediate results; the verifier then checks a small number of lightweight algebraic relations that suffice to detect any inconsistency.

The technical engine behind this shift is *proximity testing over linear error-correcting codes*. By encoding each matrix row-wise with a systematic linear code $\mathcal{C}$ of relative distance δ , the vector-matrix product $\mathbf{x} = \mathbf{rA}$ can be verified *column by column* in the encoded domain: the linearity of $\mathcal{C}$ ensures that a single inner product (“fold”) of one codeword column suffices to check one coordinate of the encoded product. If the prover cheats on any relation, the committed codewords disagree in at least a δ -fraction of positions, and a random query detects the inconsistency with probabil-

ity $\geq \delta$. Crucially, the SNARK circuit encodes the *local checks*—fold computations and Merkle path verifications at t queried positions—together with the code-commitment backend checks that bind the intermediate roots to full encoded vectors. The resulting matrix-checking circuit size is $\mathcal{O}(tk + t \log n + E_{cc}(k))$, which is *linear* in the matrix dimension when t is a small security-parameter-dependent constant and the selected backend provides $E_{cc}(k) = \mathcal{O}(k)$. This is a checking-layer result: the input commitments and the code-commitment backend are explicit components of the statement, and an end-to-end verifiable-computation system must instantiate them.

Contributions. We present **FastMatMul**, a matrix-multiplication checking layer for verifiable computation that realizes this approach. Our contributions are as follows.

- **A novel code-based verification protocol.** We design a three-round public-coin protocol that combines a structured variant of Freivalds’ randomized reduction with proximity testing over linear codes and Merkle-tree commitments. When instantiated with linear-time encodable codes (e.g., expander-based codes [5]) and a linear-time code-commitment backend with $E_{cc}(k) = \mathcal{O}(k)$, the resulting SNARK circuit size is $\mathcal{O}(tk)$ for fixed security parameter—a significant reduction from the $\mathcal{O}(k^2)$ of Freivalds-based approaches. For general constant-rate codes, backend codeword-validity checks contribute an additional $\mathcal{O}(k^2)$ term; we analyze this trade-off in Section 4.3. The theorem and the main asymptotic claim are therefore conditional on the chosen code-commitment backend satisfying the stated binding, distance, and cost assumptions; the construction should not be read as an end-to-end commitment-generation system. The protocol is made non-interactive via the Fiat–Shamir heuristic [6].
- **Rigorous security analysis.** We prove that **FastMatMul** achieves perfect completeness and computational soundness with error bounded by $\epsilon_{\text{SNARK}} + \epsilon_{cc} + \epsilon_{\text{bind}} + (k-1)/|\mathbb{F}| + 4(1-\delta)^t$, where δ is the code’s relative distance, ϵ_{SNARK} is the SNARK soundness error, ϵ_{cc} is the code-commitment backend’s codeword-binding error, and ϵ_{bind} is the Merkle commitment’s binding error. The Fiat–Shamir non-interactive variant adds the standard random-oracle loss $\epsilon_{\text{FS}}(q_H)$. The theorem is stated relative to certified public input commitments to row-wise encoded matrices and to the selected code-commitment backend, matching the committed-input viewpoint used in code-based proof systems such as Brakedown [5] and Orion [7]. We also give a concrete security-budget table that separates the reported scaling rows from a strict 128-bit deployment target. The analysis decomposes the adversary’s strategy into SNARK failure, code-commitment failure, commitment forgery, algebraic evasion, and proximity evasion, and shows each is negligible for standard parameter choices (Section 5).
- **Concrete efficiency gains.** We implement the multiplication-checking layer of **FastMatMul** in Rust

and evaluate it against a direct Freivalds SNARK baseline across matrix dimensions up to $k = 2^{13}$ under the conditional linear-time backend model. Key findings (Section 6):

- *Constraint reduction.* The implemented checking-layer constraints grow as $\mathcal{O}(k)$ under the linear-time backend model, versus $\mathcal{O}(k^2)$ for Freivalds. The gap reaches $7.5\times$ at $k = 1,024$ (419,139 vs. 3,156,298 constraints) and $30.3\times$ at $k = 4,096$ (1,667,305 vs. 50,495,818 constraints); the checking-layer implementation also scales to $k = 8,192$ with 3,331,779 constraints.
- *Proving-time speedup.* The current QA-NIZK implementation **FastMatMul**_{QA} outperforms Freivalds from $k = 256$ onward. At $k = 4,096$, prove times are 56.44 s for **FastMatMul**_{QA} and 405.04 s for Freivalds, a $7.18\times$ speedup. Proof sizes are 104 KB at $k = 4,096$ and 126 KB at $k = 8,192$.
- *GPT-2 workload case study.* For a GPT-2 workload with 36 matrix-multiplication claims at sequence length 1,024, **FastMatMul**_{QA} reduces constraints by $2.30\times$ and proving time by $1.77\times$ relative to Freivalds, while verification is slower because the current QA-NIZK implementation checks sampled opening and linking material.
- *Negligible encoding overhead.* The prover’s off-circuit encoding step takes less than 1 s even at $k = 2^{20}$, confirming that encoding does not introduce a new bottleneck.

Paper organization. Section 2 gives a high-level technical overview. Section 3 recalls the necessary background. Section 4 presents the formal construction and complexity analysis. Section 5 provides the security proof. Section 6 reports our implementation and evaluation. Section 7 surveys related work, and Section 8 concludes with future directions.

2. Technical Overview

We provide a high-level description of our protocol for verifiable matrix multiplication before presenting the formal construction in Section 4. The central challenge is to verify that a claimed product $\mathbf{C} = \mathbf{AB}$ for matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$ is correct, while keeping the verifier’s work substantially below the $\mathcal{O}(k^3)$ cost of recomputing the multiplication. Our approach combines three classical ideas—*randomized verification*, *error-correcting codes*, and *succinct commitment schemes*—in a way that decomposes the verification into components that are each efficient to check inside a SNARK circuit.

2.1. Starting Point: Freivalds’ Algorithm and Its Limitations

The classical starting point for verifying matrix multiplication is Freivalds’ algorithm [1]. Given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$ and a uniformly random vector $\mathbf{r} \in \mathbb{F}^k$, one checks

whether $\mathbf{rAB} = \mathbf{rC}$. If $\mathbf{AB} \neq \mathbf{C}$, then $\mathbf{D} := \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$, and the check $\mathbf{rD} = \mathbf{0}$ fails with probability at least $1 - k/|\mathbb{F}|$ by the Schwartz–Zippel lemma. This reduces a matrix-matrix verification to a vector-matrix computation, collapsing the problem from $\mathcal{O}(k^3)$ to $\mathcal{O}(k^2)$.

However, even the reduced $\mathcal{O}(k^2)$ cost is prohibitive when the check must be performed *inside a SNARK circuit*. In an RICS or Plonk arithmetization, each field multiplication corresponds to a constraint, so encoding the vector-matrix products $\mathbf{rA}$, $\mathbf{rB}$, and $\mathbf{rC}$ directly would still produce $\mathcal{O}(k^2)$ constraints. For matrices arising in modern deep-learning inference (where k can reach thousands), this SNARK circuit cost remains impractical.

2.2. From Vectors to Codewords: Leveraging Linear Codes

Our key observation is that the *linearity* of error-correcting codes can be exploited to push the bulk of the computation outside the circuit while retaining verifiability. Concretely, suppose we encode each row of a matrix $\mathbf{M} \in \mathbb{F}^{k \times k}$ using a systematic linear code $\mathcal{C} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ with relative minimum distance δ . Write $\text{Enc}(\mathbf{M})$ for the matrix whose i -th row is $\mathcal{C}(\mathbf{M}_{i,*})$. Because $\mathcal{C}$ is $\mathbb{F}$ -linear, taking a linear combination of codewords yields the codeword of the corresponding linear combination of messages:

$$\sum_i r_i \cdot \mathcal{C}(\mathbf{M}_{i,*}) = \mathcal{C}\left(\sum_i r_i \cdot \mathbf{M}_{i,*}\right). \quad (1)$$

In other words, the “fold” operation $\text{Fold}(\text{Enc}(\mathbf{M}), \mathbf{r}) := \sum_i r_i \cdot \text{Enc}(\mathbf{M})_{i,*}$ equals $\text{Enc}(\mathbf{rM})$ exactly. This identity is the engine that allows us to *check vector-matrix products column-by-column in the encoded domain* without materializing the full vectors.

Commitment via Merkle trees. Each encoded matrix $\text{Enc}(\mathbf{M})$ is committed column-by-column using a Merkle tree:

$$\text{cm}_{\mathbf{M}} \leftarrow \mathbf{M}.\text{CM}(\text{Enc}(\mathbf{M})).$$

The verifier starts from certified public roots $(\text{cm}_{\mathbf{A}}, \text{cm}_{\mathbf{B}}, \text{cm}_{\mathbf{C}})$ that bind the computation inputs and output at the cost of $\mathcal{O}(1)$ hash values.

2.3. Proximity-Based Verification via Random Sampling

After the Freivalds reduction, the verification task becomes: given committed encoded matrices $\text{Enc}(\mathbf{A}), \text{Enc}(\mathbf{B}), \text{Enc}(\mathbf{C})$ and a random challenge r , check that the intermediate vectors

$$\mathbf{x} = \mathbf{rA}, \quad \mathbf{y} = \mathbf{xB}, \quad \mathbf{z} = \mathbf{rC}$$

satisfy $\mathbf{y} = \mathbf{z}$. We now explain how to verify these relations in *sub-linear* time using the encoding.

The prover computes $\mathbf{x}, \mathbf{y}, \mathbf{z}$ outside the SNARK, encodes them, commits to their codewords $(\text{cm}_{\mathbf{x}}, \text{cm}_{\mathbf{y}}, \text{cm}_{\mathbf{z}})$,

and supplies the code-commitment backend material showing that these roots bind to the full encodings $\text{Enc}(\mathbf{x}), \text{Enc}(\mathbf{y}), \text{Enc}(\mathbf{z})$. The verifier then samples an ordered tuple $I = (j_1, \dots, j_t) \leftarrow_{\$} [n]^t$ of column indices independently with replacement, and for each sampled $j \in I$ requests the j -th column of every committed codeword together with its Merkle authentication path.

At each queried position j , the verifier checks four *local consistency equations*:

- (i) $\text{Enc}(\mathbf{x})_j = \text{Fold}(\text{Enc}(\mathbf{A})^{(j)}, \mathbf{r})$, ensuring $\mathbf{x} = \mathbf{rA}$;
- (ii) $\text{Enc}(\mathbf{y})_j = \text{Fold}(\text{Enc}(\mathbf{B})^{(j)}, \mathbf{x})$, ensuring $\mathbf{y} = \mathbf{xB}$;
- (iii) $\text{Enc}(\mathbf{z})_j = \text{Fold}(\text{Enc}(\mathbf{C})^{(j)}, \mathbf{r})$, ensuring $\mathbf{z} = \mathbf{rC}$;
- (iv) $\text{Enc}(\mathbf{y})_j = \text{Enc}(\mathbf{z})_j$, ensuring the Freivalds check $\mathbf{y} = \mathbf{z}$.

The crucial point is that each of these checks involves only a *single column* of the encoded matrices—an inner product of length k (for the fold operations) plus a scalar comparison. Hence the total verifier work per query is $\mathcal{O}(k)$, and for t queries the total is $\mathcal{O}(tk)$.

Why does sampling suffice? If any of the underlying vector relations fails—say $\mathbf{y} \neq \mathbf{z}$ —then the codewords $\text{Enc}(\mathbf{y})$ and $\text{Enc}(\mathbf{z})$ are distinct. By the minimum-distance property of $\mathcal{C}$, they disagree in at least a δ -fraction of positions. A uniformly random query lands on a disagreement with probability $\geq \delta$, so t independent queries detect the inconsistency except with probability at most $(1 - \delta)^t$. By a union bound over the four checked relations, the overall proximity-check failure probability is at most $4(1 - \delta)^t$. Setting $t = \lceil (\lambda + 2) / \log_2(1/(1 - \delta)) \rceil$ drives this union-bounded proximity term below $2^{-\lambda}$.

2.4. Putting It Together: The Full Protocol Pipeline

Combining the ingredients above, our protocol **FastMatMul** proceeds as follows (see Figure 1 for the formal description).

Public input. The verifier is given certified roots $(\text{cm}_{\mathbf{A}}, \text{cm}_{\mathbf{B}}, \text{cm}_{\mathbf{C}})$. They bind to row-wise encodings of $\mathbf{A}, \mathbf{B}, \mathbf{C}$ under the linear code $\mathcal{C}$.

Challenge. The verifier samples a random field element $r \in \mathbb{F}$ and sends it to the prover. This induces the structured challenge vector $\mathbf{r} = (1, r, r^2, \dots, r^{k-1})$.

Respond. The prover computes the intermediate vectors $\mathbf{x} = \mathbf{rA}$, $\mathbf{y} = \mathbf{xB}$, $\mathbf{z} = \mathbf{rC}$, encodes them, commits to the encodings $(\text{cm}_{\mathbf{x}}, \text{cm}_{\mathbf{y}}, \text{cm}_{\mathbf{z}})$, and prepares the code-commitment backend proof material for these roots.

Query. The verifier samples $I = (j_1, \dots, j_t) \leftarrow_{\$} [n]^t$ and sends the sampled indices to the prover. The prover opens the corresponding columns of all six committed codewords along with their Merkle proofs.

SNARK verification. The verifier checks the code-commitment backend relation, Merkle path validity, and the four local consistency equations at each queried index. These checks are arithmetized into the SNARK relation $\mathcal{R}_{\text{SNARK}}$ defined in Section 4. The prover then produces a succinct proof π attesting that all checks pass. The verifier only verifies π and any backend-specific linking material.

Cost summary. The *SNARK circuit size*—the key metric governing prover cost—is $\mathcal{O}(t \cdot k + t \log n + E_{cc}(k))$, where $E_{cc}(k)$ is the cost of verifying the selected code-commitment backend relation for the intermediate vectors. With a linear-time code-commitment backend, this is linear in k and independent of k^2 or k^3 . The prover additionally performs the native matrix multiplication ($\mathcal{O}(k^3)$) and encoding ($\mathcal{O}(kn)$) outside the SNARK. The verifier’s core SNARK work is $\mathcal{O}(1)$ for pairing-based backends, or $\mathcal{O}(|x|)$ for transparent ones; total verification additionally depends on the selected commitment/linking backend. A detailed cost breakdown appears in Table 3 (Section 4.3).

3. Preliminaries

We introduce notation and recall the building blocks used in our construction. Throughout, $\mathbb{F}$ denotes a finite field of size $|\mathbb{F}| = p$ for a prime p , and λ denotes the security parameter. We write $\text{negl}(\lambda)$ for any function that is negligible in λ , and PPT for probabilistic polynomial-time.

3.1. Notation

Matrices are denoted by bold uppercase letters $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$ and vectors by bold lowercase letters $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{F}^k$. For a matrix $\mathbf{M}$, we write $\mathbf{M}_{i,*}$ for its i -th row and $\mathbf{M}_{*,j}$ for its j -th column, both viewed as elements of $\mathbb{F}^k$. We write $[n] = \{1, 2, \dots, n\}$ and $I \leftarrow [n]^t$ to denote the uniform sampling of t indices from $[n]$ (with replacement). The expression $r \leftarrow \mathbb{F}$ denotes uniform random sampling from $\mathbb{F}$.

3.2. Linear Error-Correcting Codes

A *linear code* over $\mathbb{F}$ is an $\mathbb{F}$ -linear map $\mathcal{C} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ defined by a generator matrix $G \in \mathbb{F}^{k \times n}$, so that $\mathcal{C}(\mathbf{m}) = \mathbf{m} \cdot G$ for any message $\mathbf{m} \in \mathbb{F}^k$. The code has *rate* $\rho = k/n$ and *minimum distance* $d = \min_{\mathbf{c} \neq \mathbf{c}'} \Delta(\mathbf{c}, \mathbf{c}')$ over distinct codewords $\mathbf{c}, \mathbf{c}' \in \mathcal{C}$, where Δ denotes the Hamming distance. The *relative distance* is $\delta = d/n$. A code is *systematic* if the first k coordinates of $\mathcal{C}(\mathbf{m})$ equal $\mathbf{m}$ itself.

We write $\text{Enc}(\mathbf{m}) := \mathcal{C}(\mathbf{m})$ for the encoding of a message vector. For a matrix $\mathbf{M} \in \mathbb{F}^{k \times k}$, we define the *row-wise encoding* $\text{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$ as the matrix whose i -th row is $\text{Enc}(\mathbf{M}_{i,*})$. The j -th column of $\text{Enc}(\mathbf{M})$ is denoted $\text{Enc}(\mathbf{M})^{(j)} \in \mathbb{F}^k$.

The key property we exploit is *linearity*: for any coefficients $r_0, \dots, r_{k-1} \in \mathbb{F}$,

$$\sum_{i=0}^{k-1} r_i \cdot \text{Enc}(\mathbf{M}_{i,*}) = \text{Enc}\left(\sum_{i=0}^{k-1} r_i \cdot \mathbf{M}_{i,*}\right). \quad (2)$$

Definition 3.1 (Fold). For vectors $\mathbf{c}, \mathbf{r} \in \mathbb{F}^k$ with $\mathbf{c} = (c_0, \dots, c_{k-1})^\top$ and $\mathbf{r} = (r_0, \dots, r_{k-1})$, define

$$\text{Fold}(\mathbf{c}, \mathbf{r}) := \langle \mathbf{r}, \mathbf{c} \rangle = \sum_{i=0}^{k-1} r_i \cdot c_i.$$

Equivalently, for the j -th column of a row-wise encoded matrix, $\text{Fold}(\text{Enc}(\mathbf{M})^{(j)}, \mathbf{r})$ equals the j -th coordinate of $\text{Enc}(\mathbf{rM})$. This follows directly from the linearity of the code (Equation 2).

Concrete instantiations. Our protocol is compatible with any linear code. Two natural choices are: (i) *Reed–Solomon codes*, where $\mathcal{C}(\mathbf{m})$ evaluates the polynomial $\sum_i m_i X^i$ over a domain of size $n > k$, achieving the optimal (MDS) distance $\delta = 1 - k/n + 1/n$; and (ii) *expander-based codes* as used in Brakedown [5], which achieve linear-time encoding $\mathcal{O}(n)$ with constant relative distance. We discuss the trade-offs in Section 4.3.

3.3. Merkle Tree Commitment

A *Merkle tree* [8] over a collision-resistant hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ provides a vector commitment scheme with the following algorithms:

- $\text{cm} \leftarrow \text{M.CM}(\mathbf{v})$: Given a vector $\mathbf{v} = (v_1, \dots, v_n)$, construct a binary hash tree with leaves $H(v_1), \dots, H(v_n)$ and output the root hash cm .
- $(v_j, \pi_j) \leftarrow \text{Merkle.Open}(\text{cm}, j)$: Open position j , returning the leaf value v_j and the authentication path π_j (the sequence of sibling hashes from the leaf to the root).
- $b \leftarrow \text{Merkle.Verify}(\pi_j, v_j, \text{cm})$: Verify that v_j is the j -th leaf consistent with root cm ; output $b \in \{0, 1\}$.

The scheme is *position-binding*: under the collision resistance of H , no PPT adversary can produce (j, v, π) and (j, v', π') with $v \neq v'$ such that both verify against the same root cm , except with probability $\epsilon_{\text{bind}}(\lambda) = \text{negl}(\lambda)$.

3.4. Succinct Non-Interactive Arguments of Knowledge

A *SNARK* (Succinct Non-interactive Argument of Knowledge) for an NP relation $\mathcal{R}$ consists of algorithms ($\text{Setup}, \text{SNARK.P}, \text{SNARK.V}$):

- $\text{pp} \leftarrow \text{Setup}(1^\lambda, \mathcal{R})$: Generate public parameters.
- $\pi \leftarrow \text{SNARK.P}(\text{pp}, \mathbf{x}, \mathbf{w})$: Given a statement $\mathbf{x}$ and witness $\mathbf{w}$ with $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, produce a proof π .
- $b \leftarrow \text{SNARK.V}(\text{pp}, \mathbf{x}, \pi)$: Verify the proof and output $b \in \{0, 1\}$.

A SNARK satisfies *completeness* (an honest prover always convinces the verifier), *knowledge soundness* (any PPT prover that makes the verifier accept can be used to extract a valid witness), and *succinctness* (the proof size $|\pi|$ and verifier time are $\text{poly}(\lambda, |x|)$, independent of the witness size and computation).

Our protocol is modular with respect to the choice of SNARK backend. The key metric is the *circuit size* (number of RICS constraints or PlonK gates) of the relation $\mathcal{R}_{\text{SNARK}}$ that the backend must prove, as this dominates the prover’s cost.

3.5. Freivalds' Algorithm

Freivalds' algorithm [1] is a classical randomized algorithm for verifying matrix multiplication. Given $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$, it samples $\mathbf{r} \leftarrow \mathbb{F}^k$ and checks whether $\mathbf{r}(\mathbf{AB}) = \mathbf{rC}$. The correctness relies on the following:

Lemma 3.2 (Schwartz–Zippel [9], [10]). *Let $P(X_1, \dots, X_m)$ be a non-zero polynomial of total degree d over a finite field $\mathbb{F}$. Then for $r_1, \dots, r_m \leftarrow \mathbb{F}$ chosen uniformly and independently,*

$$\Pr[P(r_1, \dots, r_m) = 0] \leq \frac{d}{|\mathbb{F}|}.$$

In our protocol, we use a structured variant: instead of sampling $\mathbf{r} \leftarrow \mathbb{F}^k$ uniformly, we sample a single $r \leftarrow \mathbb{F}$ and set $\mathbf{r} = (1, r, r^2, \dots, r^{k-1})$. If $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$, then for any non-zero column $\mathbf{d}_j$ of $\mathbf{D}$, the inner product $\langle \mathbf{r}, \mathbf{d}_j \rangle = \sum_{i=0}^{k-1} (\mathbf{D})_{i,j} \cdot r^i$ is a non-zero univariate polynomial in r of degree at most $k-1$. By Lemma 3.2, this evaluates to zero with probability at most $(k-1)/|\mathbb{F}|$.

The Vandermonde structure offers a concrete efficiency advantage beyond mere randomness reduction. The challenge r appears in the SNARK public statement $\mathbf{x}$ (Section 4); using a single scalar keeps $|\mathbf{x}| = \mathcal{O}(1)$ field elements, independent of k . Had we used a uniformly random $\mathbf{r} \in \mathbb{F}^k$, the statement size would grow to $\mathcal{O}(k)$, increasing verifier time for statement-size-sensitive SNARKs (e.g., transparent SNARKs where verifier time is $\mathcal{O}(|\mathbf{x}|)$) and defeating the goal of a k -independent verifier cost. The soundness bound $(k-1)/|\mathbb{F}|$ is slightly weaker than the $1/|\mathbb{F}|$ bound of a fully random $\mathbf{r}$, but remains negligible for any $|\mathbb{F}| \geq 2^{256}$ and practical k .

4. Our Construction

In this section we present the formal construction of our verifiable matrix multiplication protocol **FastMatMul**. We begin by defining the relation chain that reduces the original matrix multiplication statement to a succinct SNARK-friendly relation (Section 4.1), then give the full protocol description (Section 4.2), and conclude with a complexity analysis (Section 4.3). Table 1 summarizes the notation used throughout the paper.

4.1. Relation Reduction Chain

Our construction proceeds through two successive reductions. Each reduction preserves completeness exactly and introduces a bounded soundness error that we analyze in Section 5.

Input-commitment model. Following the committed-oracle viewpoint common in code-based proof systems such as Brakedown [5] and Orion [7], we take the input matrix commitments as certified public inputs. Concretely, $\text{cm}_A, \text{cm}_B, \text{cm}_C$ are assumed to bind to row-wise encodings $\text{Enc}(\mathbf{A}), \text{Enc}(\mathbf{B}), \text{Enc}(\mathbf{C})$ of some matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$. Certification means that every accepting opening of

Symbol	Description
<i>Parameters</i>	
$\mathbb{F}$	Finite field
k	Matrix dimension ($k \times k$)
n	Codeword length of $\mathcal{C}$
δ	Relative minimum distance of $\mathcal{C}$
$t = I $	Query complexity (number of sampled indices)
λ	Security parameter
<i>Matrices and Vectors</i>	
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	Input/output matrices in $\mathbb{F}^{k \times k}$
$\mathbf{D}$	Error matrix $\mathbf{AB} - \mathbf{C}$
$\mathbf{r}$	Challenge vector $(1, r, r^2, \dots, r^{k-1})$
$\mathbf{x}, \mathbf{y}, \mathbf{z}$	Intermediate vectors: $\mathbf{rA}, \mathbf{xB}, \mathbf{rC}$
<i>Encoding and Commitment</i>	
$\mathcal{C}$	Linear error-correcting code $\mathbb{F}^k \rightarrow \mathbb{F}^n$
$\text{Enc}(\cdot)$	Encoding function (row-wise for matrices)
$\text{Enc}(\mathbf{M})^{(j)}$	j -th column of $\text{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$
$\text{Fold}(\mathbf{c}, \mathbf{r})$	Inner product $\langle \mathbf{r}, \mathbf{c} \rangle$
$\text{M.CM}(\cdot)$	Merkle tree commitment
$\text{Merkle.Open}(\text{cm}, j)$	Merkle tree opening at index j
$\text{Merkle.Verify}(\pi, v, \text{cm})$	Merkle tree verification
$\text{Open.Verify}(\pi, v, \text{cm}, j)$	Position-opening verifier of selected backend
cm_χ	Merkle root for committed object χ
$\pi_{\chi,j}$	Merkle authentication path for χ at position j
σ_v	Code-commitment backend proof/witness for cm_v
<i>Sets and tuples</i>	
$\mathcal{M}$	Matrix set $\{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$
$\mathcal{W}$	Vector set $\{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$
$\mathcal{S}$	Full committed-object set $\mathcal{M} \cup \mathcal{W}$
I	Ordered tuple of queried column indices ($I \in [n]^t$)
<i>SNARK</i>	
$\mathbf{x}$	SNARK public statement (instance)
$\mathbf{w}$	SNARK witness
π	SNARK proof
$\text{SNARK.P}, \text{SNARK.V}$	SNARK prover and verifier
pp	Public parameters

TABLE 1. SUMMARY OF NOTATION.

cm_M , for $M \in \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$, is a coordinate or column of $\text{Enc}(M)$ for a fixed matrix M determined before the verifier samples the Freivalds challenge. This certification may be supplied by the surrounding committed-codeword system, by a public generation process, or by a separate input-opening proof. It is a precondition of the multiplication-checking protocol rather than an online cost charged to **FastMatMul**. The goal of **FastMatMul** is to verify the multiplication relation among these committed matrices without arithmetizing the full matrix product. Throughout this section, $\text{M.CM}(\text{Enc}(\mathbf{M}))$ for an encoded matrix means the Merkle root over the length- n sequence of encoded columns $(\text{Enc}(\mathbf{M})^{(1)}, \dots, \text{Enc}(\mathbf{M})^{(n)})$. For an encoded vector, $\text{M.CM}(\text{Enc}(v))$ commits to the length- n sequence of coordinates.

Definition 4.1 (Code-commitment backend). Fix a systematic linear code $\mathcal{C} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ and a position-opening layer whose verification predicate is written $\text{Open.Verify}(\pi_j, a, \text{cm}, j)$. For the Merkle instantiation, Open.Verify is exactly Merkle.Verify with the queried index included in the leaf domain; for KZG or CPLink-style instantiations it is the corresponding opening or linking verifier. A code-commitment backend **CodeCom** provides a prover relation **CodeCom.Prove** and a verifier relation

CodeCom.Verify for vector commitments. It must satisfy the following two properties.

Completeness. For every $v \in \mathbb{F}^k$, if cm_v commits to $\text{Enc}(v) = \mathcal{C}(v)$ under the chosen opening layer, then an honest prover can produce proof material $\sigma_v \leftarrow \text{CodeCom.Prove}(\text{cm}_v, v, \text{Enc}(v))$ accepted by **CodeCom.Verify**(cm_v, v, σ_v).

Global codeword binding. The backend is codeword-binding with error $\epsilon_{\text{cc}}(\lambda)$ if, for every PPT adversary $\mathcal{A}$,

$$\Pr[\text{Bad}_{\text{cc}}^{\mathcal{A}}(\lambda) = 1] \leq \epsilon_{\text{cc}}(\lambda).$$

The event $\text{Bad}_{\text{cc}}^{\mathcal{A}}$ occurs if $\mathcal{A}$ outputs $(\text{cm}_v, v, \sigma_v, j, a, \pi_j)$ such that **CodeCom.Verify**(cm_v, v, σ_v) = 1, **Open.Verify**(π_j, a, cm_v, j) = 1, and $a \neq \text{Enc}(v)_j$. Thus, except with probability ϵ_{cc} , any vector commitment accepted by **CodeCom.Verify** is globally linked to the full codeword $\text{Enc}(v)$: every accepting opening at every coordinate is forced to equal the corresponding coordinate of $\text{Enc}(v)$, not merely the sampled coordinates. The separate position-binding error of the opening layer is accounted for by ϵ_{bind} in Theorem 5.1. The cost of verifying **CodeCom.Verify**(cm_v, v, σ_v) is denoted $E_{\text{cc}}(k)$.

We do not introduce a new commitment primitive. Instead, **FastMatMul** is stated relative to this committed-codeword interface. This matches the role played by encoded committed oracles in Brakedown [5] and Orion [7]. Operationally, the backend can be instantiated by a Brakedown/Orion-style code-commitment layer, an in-circuit encoder check tied to the committed root, or a commitment-to-codeword linking proof supplied by the chosen backend.

Backend contract used by the theorem. Theorem 5.1 uses only the following backend facts: (i) the input matrix roots are certified commitments to fixed row-wise codewords before r is sampled; (ii) the intermediate vector roots are sent before I is sampled; (iii) an accepting **CodeCom.Verify**(cm_v, v, σ_v) globally links cm_v to $\text{Enc}(v)$ as in Definition 4.1; and (iv) the opening layer is position-binding for every queried coordinate. No part of the proof relies on a new algebraic assumption specific to **FastMatMul** beyond this committed-codeword interface.

Backend boundary. The security theorem is compositional: it proves the multiplication-checking protocol relative to any backend satisfying Definition 4.1. An end-to-end system must choose one concrete backend and specify its algorithms, the contents of σ_v , and the constraint/proof-size contribution of **CodeCom.Verify**. Our implementation evaluates the local fold/opening/linking layer under a linear-time $E_{\text{cc}}(k) = \mathcal{O}(k)$ backend model; it does not yet implement or measure **CodeCom.Verify** independently. Table 2 makes this boundary explicit.

The original relation $\mathcal{R}_{\text{orig}}$. The starting point is the statement that the prover knows matrices whose *encoded* commitments satisfy the multiplication relation. The public

instance is $(\text{cm}_A, \text{cm}_B, \text{cm}_C)$ and the witness is $(\mathbf{A}, \mathbf{B}, \mathbf{C})$; the relation $\mathcal{R}_{\text{orig}}$ requires:

$$\begin{aligned} \text{cm}_A &= \text{M.CM}(\text{Enc}(\mathbf{A})), & \text{cm}_B &= \text{M.CM}(\text{Enc}(\mathbf{B})), \\ \text{cm}_C &= \text{M.CM}(\text{Enc}(\mathbf{C})), & \mathbf{AB} &= \mathbf{C}. \end{aligned}$$

Thus a matrix commitment binds the verifier to the row-wise codeword matrix rather than to the raw matrix alone. Since $\mathcal{C}$ is systematic, the raw matrix is also determined by the first k coordinates of each committed row. Proving membership in $\mathcal{R}_{\text{orig}}$ directly inside a SNARK would require $\mathcal{O}(k^3)$ multiplication constraints, which is exactly what we aim to avoid.

Reduction 1: Freivalds' randomization ($\mathcal{R}_{\text{orig}} \rightsquigarrow \mathcal{R}_{\text{Freivalds}}$). Using the verifier's random challenge $r \in \mathbb{F}$, we construct the vector $\mathbf{r} = (1, r, r^2, \dots, r^{k-1})$ and define intermediate vectors $\mathbf{x} = \mathbf{rA}$, $\mathbf{y} = \mathbf{xB}$, $\mathbf{z} = \mathbf{rC}$. Let $\mathcal{S} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{x}, \mathbf{y}, \mathbf{z}\}$ denote the full set of committed objects. The instance is $(\{\text{cm}_\chi\}_{\chi \in \mathcal{S}}, r)$ and the witness is $\{\chi\}_{\chi \in \mathcal{S}}$; the relation $\mathcal{R}_{\text{Freivalds}}$ requires:

$$\begin{aligned} \bigwedge_{\chi \in \mathcal{S}} \text{cm}_\chi &= \text{M.CM}(\text{Enc}(\chi)), \\ \mathbf{rA} &= \mathbf{x} \wedge \mathbf{xB} = \mathbf{y}, \\ \mathbf{rC} &= \mathbf{z} \wedge \mathbf{y} = \mathbf{z}. \end{aligned}$$

By the Schwartz–Zippel lemma, if $\mathbf{AB} \neq \mathbf{C}$, then $\mathbf{y} \neq \mathbf{z}$ with probability at least $1 - (k-1)/|\mathbb{F}|$. Note that while this relation eliminates the $\mathcal{O}(k^3)$ matrix product, verifying the vector-matrix products still costs $\mathcal{O}(k^2)$ each.

Reduction 2: Proximity lifting ($\mathcal{R}_{\text{Freivalds}} \rightsquigarrow \mathcal{R}_{\text{SNARK}}$). We now encode all matrices and vectors using a systematic $\mathbb{F}$ -linear code $\mathcal{C} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ with relative distance δ , and replace the global vector-matrix checks with *local* consistency checks at random positions. Let $I = (j_1, \dots, j_t) \leftarrow \$_{[n]}^t$ be the ordered tuple of queried column indices sampled independently with replacement, and let $\mathcal{M} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$, $\mathcal{W} = \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$ denote the matrix set and vector set respectively.

The *public instance* is $\mathbf{x} = (\{\text{cm}_\chi\}_{\chi \in \mathcal{S}}, r, I)$ and the *witness* is

$$\mathbf{w} = (\mathbf{x}, \mathbf{y}, \mathbf{z}, \sigma_x, \sigma_y, \sigma_z, \{\text{Enc}_\chi^{(j)}, \pi_{\chi,j}\}_{j \in I, \chi \in \mathcal{S}}).$$

For the path check, write $e_{\chi,j} := \text{Enc}_\chi^{(j)}$ for the opened value. The relation $\mathcal{R}_{\text{SNARK}}$ requires $(\mathbf{x}, \mathbf{w})$ to satisfy all of:

$$\begin{aligned} &\text{CodeCom.Verify}(\text{cm}_x, \mathbf{x}, \sigma_x) = 1 \wedge \text{CodeCom.Verify}(\text{cm}_y, \mathbf{y}, \sigma_y) = 1 \\ &\quad \wedge \text{CodeCom.Verify}(\text{cm}_z, \mathbf{z}, \sigma_z) = 1, & (\text{CODECOM}) \\ &\forall j \in I: \text{Enc}_{x,j} = \text{Fold}(\text{Enc}_A^{(j)}, \mathbf{r}), & (\text{FOLD-A}) \\ &\forall j \in I: \text{Enc}_{y,j} = \text{Fold}(\text{Enc}_B^{(j)}, \mathbf{x}), & (\text{FOLD-B}) \\ &\forall j \in I: \text{Enc}_{z,j} = \text{Fold}(\text{Enc}_C^{(j)}, \mathbf{r}), & (\text{FOLD-C}) \\ &\forall j \in I: \text{Enc}_{y,j} = \text{Enc}_{z,j}, & (\text{EQ}) \\ &\forall \chi \in \mathcal{S}, j \in I: \text{Open.Verify}(\pi_{\chi,j}, e_{\chi,j}, \text{cm}_\chi, j) = 1 & (\text{PATH}) \end{aligned}$$

Here $\forall j \in I$ ranges over every sampled position in the ordered tuple, including repetitions. For a matrix commitment, $\text{Enc}_\chi^{(j)}$ denotes the full j -th encoded column; for a vector commitment, it denotes the scalar coordinate $\text{Enc}(\chi)_j$. Equation (PATH) uses the selected position-opening verifier. For the transparent Merkle instantiation,

Backend choice	Setup / assumption	$E_{cc}(k)$	Role in this paper
Dense encoder check	Transparent opening binding; explicit generator matrix	$\mathcal{O}(k(n-k))$	Sound but quadratic at constant rate; analyzed as a fallback backend cost, not used for the primary asymptotic claim.
Linear-time committed-codeword backend	Backend certifies distance δ and global codeword binding	$\mathcal{O}(k)$ at constant rate	Target model for the $\mathcal{O}(tk)$ theorem; an end-to-end artifact must instantiate and measure it.
Algebraic linking backend	Backend-specific setup and binding assumption	Backend dependent	Compatible with the relation, but its proof size and verifier cost must be analyzed separately.
Current QA artifact	Merkle openings plus QA-NIZK linking for sampled columns	Measures local layer only	Reports fold/opening/linking costs; standalone CodeCom.Verify cost is not independently implemented or measured.

TABLE 2. BACKEND BOUNDARY FOR FASTMATMUL. THE SECURITY THEOREM IS RELATIVE TO A BACKEND SATISFYING DEFINITION 4.1; THE CURRENT ARTIFACT MEASURES THE MULTIPLICATION-CHECKING LAYER AND NOT A CLOSED END-TO-END BACKEND.

this is **Merkle.Verify** with the queried index included in the leaf domain. In KZG or CPLink-style variants, the same slot is occupied by the selected opening or linking verifier; the security proof uses only position binding of the opening layer and the codeword-binding property of Definition 4.1.

The critical property is that $\mathcal{R}_{\text{SNARK}}$ involves only $\mathcal{O}(|I| \cdot k)$ constraints for the fold operations, plus $\mathcal{O}(|I| \cdot \log n)$ hash checks for the Merkle paths and the backend code-commitment cost $E_{cc}(k)$. Since $|I|$ is a security-parameter-sized constant independent of k , the circuit size is *linear* in k .

Codeword-validity cost. The backend checks in (CODECOM) force the prover to commit to *full* valid vector codewords before the query tuple is sampled. Without this binding step, a cheating prover could choose sampled openings that satisfy the fold checks without committing to any genuine encoding of a fixed vector, defeating the distance-based soundness argument. Analogously, the input matrix commitments $\text{cm}_A, \text{cm}_B, \text{cm}_C$ must be well-formed commitments to row-wise encodings of $\mathbf{A}, \mathbf{B}, \mathbf{C}$. In our model this is part of the public input-commitment interface, not a property re-proved by the multiplication protocol itself. For a general linear code represented by a dense generator matrix $G \in \mathbb{F}^{k \times n}$, the constraint $\text{Enc}_v = v \cdot G$ requires $\mathcal{O}(k(n-k))$ inner-product work. For linear-time encodable codes (e.g., expander-based codes [5]) and a code-commitment backend whose verifier follows the encoder circuit, the same validity check can be arithmetized with $\mathcal{O}(n)$ constraints, which is $\mathcal{O}(k)$ at constant rate. Thus the backend codeword-validity term is linear for the code family used in Table 5, but becomes $\mathcal{O}(k^2)$ for dense general constant-rate codes. We discuss the trade-off in Section 4.3.

4.2. Protocol Description

We now give the full specification of **FastMatMul**. Let $\mathcal{C} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a systematic linear code with relative distance δ and M.CM denote the Merkle tree commitment. Let $\mathcal{M} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$ be the matrix set, $\mathcal{W} = \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$ the vector set, and $\mathcal{S} = \mathcal{M} \cup \mathcal{W}$. The verifier is given certified input commitments $(\text{cm}_A, \text{cm}_B, \text{cm}_C)$ to $\text{Enc}(\mathbf{A}), \text{Enc}(\mathbf{B}), \text{Enc}(\mathbf{C})$. The protocol **FastMatMul** operates between a prover $\mathcal{P}$ and verifier $\mathcal{V}$ as shown in Figure 1.

Remark on the SNARK relation. The SNARK proof π attests to the validity of $\mathcal{R}_{\text{SNARK}}$ defined in Section 4.1. The circuit verifies three groups of constraints. First, constraint (CODECOM) ensures that $\text{cm}_x, \text{cm}_y, \text{cm}_z$ bind, via the selected code-commitment backend, to the full encodings of the same $\mathbf{x}, \mathbf{y}, \mathbf{z}$ used in the fold checks. Second, constraints (FOLD-A)–(EQ) verify the four fold/equality checks at each $j \in I$. Third, constraint (PATH) checks consistency of all Merkle opening proofs with the committed roots. The public statement $\mathbf{x}$ consists of the commitment roots $\{\text{cm}_\chi\}_{\chi \in \mathcal{S}}$, the challenge r , and the query tuple I . The witness $\mathbf{w}$ consists of the intermediate vectors, the backend proofs/witnesses for their codeword commitments, and the opened codeword columns with their authentication paths.

Non-interactive variant. The protocol as described is a three-round public-coin interactive protocol. By applying the Fiat–Shamir heuristic [6] in the random oracle model, it can be made non-interactive using domain-separated transcript hashes:

$$r \leftarrow \text{Hash}(\text{FMM-}r, \text{pp}, \text{cm}_A, \text{cm}_B, \text{cm}_C),$$

and

$$\begin{aligned} T_I &\leftarrow (\text{pp}, \text{cm}_A, \text{cm}_B, \text{cm}_C, r, \text{cm}_x, \text{cm}_y, \text{cm}_z), \\ I &\leftarrow \text{Sample}_t(\text{Hash}(\text{FMM-I}, T_I)). \end{aligned}$$

The resulting non-interactive proof consists of the commitment roots, the transcript-derived challenges, and the SNARK proof π ; backend-specific opening, linking, or backend code-commitment material is either carried as part of π or as auxiliary proof data verified by the corresponding backend.

Security of the non-interactive variant. A potential concern with the Fiat–Shamir transform for proximity-based protocols is a *grinding attack*: a malicious prover could try many transcript prefixes until the derived challenge r or query tuple I is favorable. The random-oracle theorem accounts for this by the standard $\epsilon_{\text{FS}}(q_H)$ loss in Theorem 5.1. For intuition, consider grinding only the intermediate roots after r is fixed. Suppose the adversary has committed to $(\text{cm}_A, \text{cm}_B, \text{cm}_C)$ (and hence to fixed encoded matrices) and, after receiving r , is searching for $(\text{cm}_x, \text{cm}_y, \text{cm}_z)$ such that the transcript-derived query tuple I avoids the disagreement set D of some fold or equality check. If

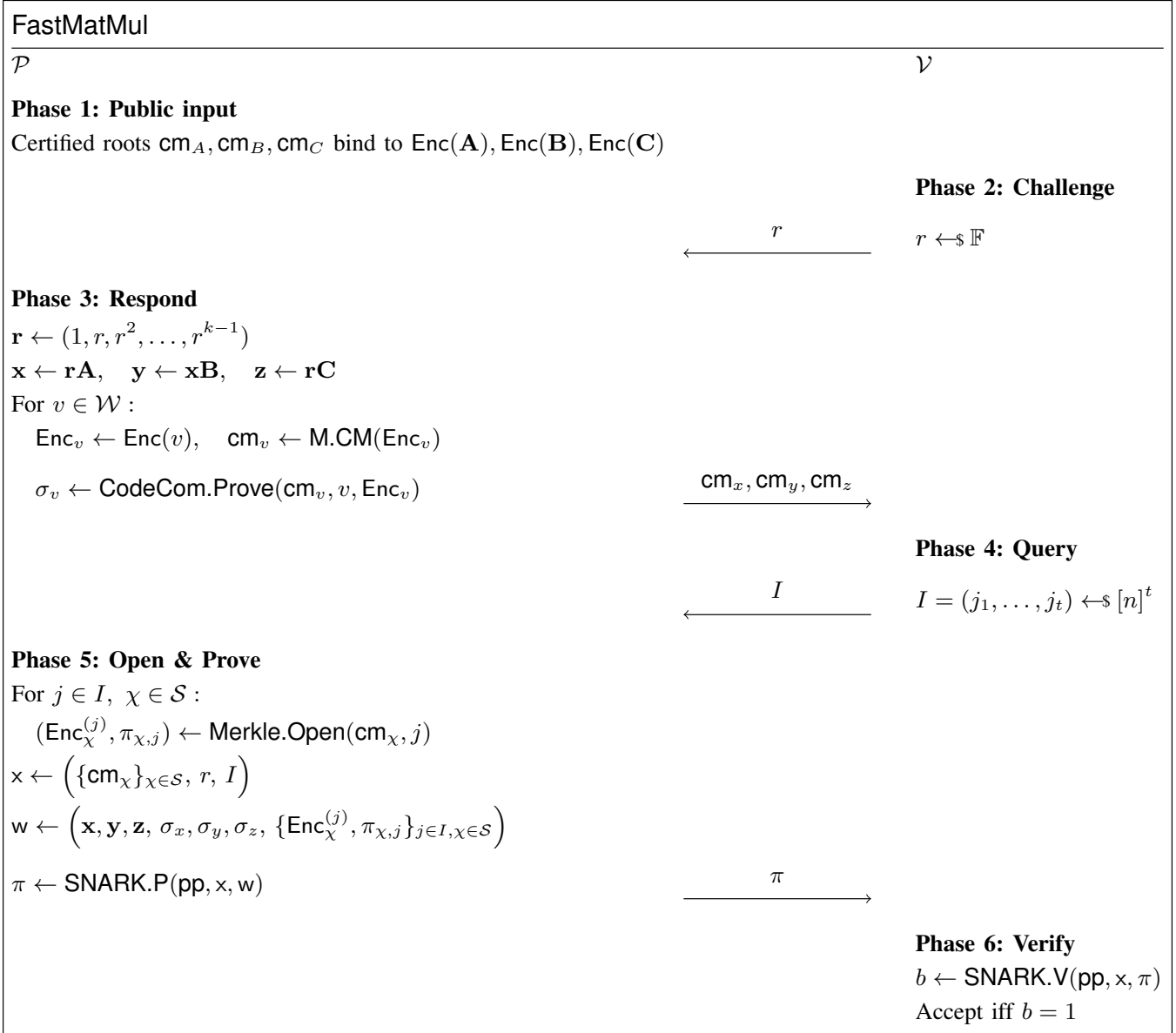


Figure 1. The **FastMatMul** protocol for verifiable matrix multiplication. $\mathcal{S} = \mathcal{M} \cup \mathcal{W}$ is the full set of committed objects, $t = |I|$ is the query complexity, and $\mathbf{x}, \mathbf{w}$ denote the public statement and witness for the inner SNARK.

any of the four checks fails globally, then $|D| \geq \delta n$ by the minimum-distance property. For a single random query $j \in [n]$, the probability of $j \notin D$ is at most $1 - \delta$; for t independent queries it is $(1 - \delta)^t$. Choosing t as in Section 5 makes the union-bounded proximity term $4(1 - \delta)^t$ at most $2^{-\lambda}$. Each trial of the grinding attack produces an independently random I in the random oracle model because the query hash binds the full transcript, so the probability that q grinding attempts yield a favorable I is at most $q \cdot 4(1 - \delta)^t$. For any PPT adversary, q is polynomially bounded in λ , giving a total success probability of $\text{negl}(\lambda)$. Hence the non-interactive protocol retains the soundness bound of Theorem 5.1 in the random oracle model, plus the stated Fiat-Shamir loss.

Zero-knowledge extension. The protocol as stated is a *verifiable computation* protocol and does not conceal the inputs $\mathbf{A}, \mathbf{B}, \mathbf{C}$ from the verifier. Zero-knowledge can be achieved straightforwardly by two modifications: (i) replace the Merkle tree commitments with a *hiding* commitment scheme (e.g., salted hashes or Pedersen vector commitments [11]) so that the committed encoded matrices reveal no information about $\mathbf{A}, \mathbf{B}, \mathbf{C}$; and (ii) employ a zero-knowledge SNARK backend for the inner proof π . Masking the intermediate vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$ requires adding fresh random blinding vectors in Phase 3 and including the corresponding decommitment randomness in the SNARK witness. We omit a full ZK construction and proof here; hiding the sampled openings, the backend code-commitment

material, and any linking material requires a separate simulation argument.

4.3. Complexity Analysis

We analyze the costs of **FastMatMul**.

Prover computation. The prover’s dominant costs are:

- (i) *Matrix multiplication*: Computing $\mathbf{C} = \mathbf{AB}$ requires $\mathcal{O}(k^3)$ field operations (or $\mathcal{O}(k^\omega)$ with fast algorithms). This is performed natively outside the SNARK circuit and is amenable to GPU acceleration.
- (ii) *Encoding*: $3k$ rows (one per matrix row, three matrices) each encoded into length- n codewords costs $\mathcal{O}(kn)$ per matrix with linear-time codes.
- (iii) *Vector computation*: Computing $\mathbf{x}, \mathbf{y}, \mathbf{z}$ costs $\mathcal{O}(k^2)$ in total.
- (iv) *SNARK proving*: The circuit size is $\mathcal{O}(t \cdot k + t \cdot \log n)$ for the fold and Merkle checks, plus the backend codeword-validity term $E_{cc}(k)$ (see below). The SNARK prover time scales with the circuit size.

Verifier computation. The verifier performs SNARK verification together with any commitment/linking checks required by the chosen backend. For a pairing-based SNARK whose public input consists only of the roots, challenge, and query indices, the core SNARK verification is $\mathcal{O}(1)$. For transparent or linking-heavy backends, verification may scale with the public input or linking proof size; the concrete behavior is reported in Section 6.

Communication. The communication depends on the commitment layer. With a fully succinct vector-commitment backend, the proof consists of a constant number of commitment digests plus the SNARK proof, giving $\mathcal{O}(\lambda)$ communication. With the QA-NIZK/Merkle implementation evaluated in this paper, the proof also contains authentication and linking material for the queried encoded columns. The sampled-column material scales as $\mathcal{O}(tk + t \log n)$ field/hash elements for fixed security parameter; any additional backend material is backend dependent and accounted for in the concrete proof sizes. This trade-off explains the sub-megabyte proof sizes reported in Section 6 and Appendix C.

Circuit size comparison. Table 3 summarizes the asymptotic costs. Naïve arithmetization uses $\mathcal{O}(k^3)$ constraints. Freivalds’ algorithm alone uses $\mathcal{O}(k^2)$. Our approach achieves

$$\mathcal{O}(tk + t \log n + E_{cc}(k)) :$$

the first two terms cover the fold and Merkle checks, while $E_{cc}(k)$ denotes the cost of verifying the code-commitment backend relation for the intermediate vector commitments. For dense general codes, $E_{cc}(k) = \mathcal{O}(k(n - k))$, which is $\mathcal{O}(k^2)$ at constant rate. For linear-time encodable codes of rate ρ and distance δ , $E_{cc}(k) = \mathcal{O}(k)$ when the selected code-commitment backend has a linear-time verifier. Setting

$$t = \left\lceil \frac{\lambda + 2}{\log_2(1/(1 - \delta))} \right\rceil$$

Component	Prover	Verifier
Matrix multiplication	$\mathcal{O}(k^3)$	—
Encoding (row-wise)	$\mathcal{O}(kn)$	—
Merkle commitment	$\mathcal{O}(kn)$	—
SNARK circuit size	$\mathcal{O}(tk + t \log n + E_{cc}(k))$	—
Core SNARK verification	—	$\mathcal{O}(1)$
Total	$\mathcal{O}(k^3 + kn)$	backend-dependent

TABLE 3. ASYMPTOTIC COST COMPARISON. k : MATRIX DIMENSION; n : CODEWORD LENGTH; t : QUERY COUNT; $E_{cc}(k)$: COST OF THE SELECTED CODE-COMMITMENT BACKEND RELATION.

then gives a total circuit size of $\mathcal{O}(k)$ for fixed security parameter, with concrete sizes discussed in Section 6.

5. Security Analysis

We prove that the protocol **FastMatMul** (Figure 1) constitutes a secure interactive argument for the matrix multiplication relation $\mathcal{R}_{\text{orig}}$ relative to a code-commitment backend and certified encoded input commitments. Soundness and completeness are established in Theorem 5.1 below; knowledge soundness is discussed separately in Remark 2. As discussed in Section 4.1, the public input commitments $\text{cm}_A, \text{cm}_B, \text{cm}_C$ are assumed to bind to row-wise encodings of fixed matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$. This is the committed-input model used by code-based proof systems such as Brakedown [5] and Orion [7]; **FastMatMul** verifies the multiplication relation among these committed objects. For the intermediate vectors, the SNARK relation invokes the selected code-commitment backend relation $\text{CodeCom.Verify}(\text{cm}_v, v, \sigma_v)$, ensuring that the commitments $\text{cm}_x, \text{cm}_y, \text{cm}_z$ bind to full codewords $\text{Enc}(\mathbf{x}), \text{Enc}(\mathbf{y}), \text{Enc}(\mathbf{z})$ before the query tuple is sampled.

Soundness-critical ordering. The proof uses the following timing invariants. First, the certified input roots $\text{cm}_A, \text{cm}_B, \text{cm}_C$ are fixed before the Freivalds challenge r is sampled. Second, the intermediate roots $\text{cm}_x, \text{cm}_y, \text{cm}_z$ are fixed before the query tuple $I = (j_1, \dots, j_t) \leftarrow [n]^t$ is sampled; conditioned on an accepting backend proof, each such root is globally linked to one full vector codeword independently of the sampled positions. Third, the entries of I are sampled independently with replacement, as defined in Section 3.1. If an implementation samples without replacement, the proximity failure probability only decreases; the theorem states the simpler with-replacement bound.

Theorem 5.1 (Security). *The protocol **FastMatMul** satisfies the following properties:*

- (i) **Perfect completeness.** *If $\mathbf{AB} = \mathbf{C}$, an honest prover always makes the verifier accept.*
- (ii) **Interactive computational soundness.** *For any PPT adversary $\mathcal{P}^*$ given certified public input commitments*

to $\mathbf{A}, \mathbf{B}, \mathbf{C}$, let Acc be the event that the verifier accepts a false committed matrix statement. Then:

$$\Pr[\text{Acc} \mid \mathbf{AB} \neq \mathbf{C}] \leq \epsilon_{\text{SNARK}}(\lambda) + \epsilon_{\text{cc}}(\lambda) + \epsilon_{\text{bind}}(\lambda) + \frac{k-1}{|\mathbb{F}|} + 4(1-\delta)^t,$$

Here $t = |I|$ is the query complexity and δ is the relative minimum distance of $\mathcal{C}$. The terms ϵ_{SNARK} , ϵ_{cc} , and ϵ_{bind} denote, respectively, the soundness error of the SNARK backend, the global codeword-binding error of the selected code-commitment backend, and the union-bounded position-binding error of the opening layer over all roots and openings used in the protocol.

- (iii) **Fiat-Shamir soundness.** In the random oracle model, the non-interactive variant described in Section 4.2 satisfies the same bound with an additional Fiat-Shamir loss term $\epsilon_{\text{FS}}(q_H)$, where q_H is the adversary's number of random-oracle queries.

Table 4 spells out the concrete interpretation of the bound. The table separates the reported scaling experiments, which use $t = 128$, from a strict 128-bit deployment certificate. The latter requires a concrete backend instantiation that certifies δ , the global codeword-binding error, and the position-opening binding error.

Proof sketch. Completeness is immediate from linearity: if $\mathbf{AB} = \mathbf{C}$, then for every Freivalds challenge $\mathbf{r}$ the honest vectors satisfy $\mathbf{x} = \mathbf{rA}$, $\mathbf{y} = \mathbf{xB}$, $\mathbf{z} = \mathbf{rC}$, and hence $\mathbf{y} = \mathbf{z}$. Linearity of the code makes every sampled fold identity hold, and the honest commitment/opening proofs verify.

For soundness, condition on the complement of three backend failures: SNARK soundness, global codeword binding for the intermediate roots, and position-binding for accepted openings. These contribute $\epsilon_{\text{SNARK}}(\lambda) + \epsilon_{\text{cc}}(\lambda) + \epsilon_{\text{bind}}(\lambda)$. Under this conditioning, the certified input roots determine fixed matrices before r is sampled, and each accepting intermediate root determines one full codeword before the sampled query tuple I is sampled. If $\mathbf{AB} \neq \mathbf{C}$, then for $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$ the equation $\mathbf{rD} = \mathbf{0}$ is a nonzero univariate polynomial test of degree at most $k-1$, so Schwartz-Zippel bounds the Freivalds failure by $(k-1)/|\mathbb{F}|$. Conditioned further on $\mathbf{rD} \neq \mathbf{0}$, no vectors can satisfy all four global equalities $\mathbf{x} = \mathbf{rA}$, $\mathbf{y} = \mathbf{xB}$, $\mathbf{z} = \mathbf{rC}$, and $\mathbf{y} = \mathbf{z}$. At least one corresponding pair of codewords is therefore distinct, and the relative distance δ makes a random queried coordinate reject with probability at least δ . Taking t independent sampled coordinates and union-bounding over the four relations gives $4(1-\delta)^t$. The Fiat-Shamir statement follows by the standard random-oracle loss for the transcript-derived challenge and query tuple. Appendix A gives the detailed proof. $\square$

Remark 1 (Fiat-Shamir soundness). The proof of the main bound in Theorem 5.1 is for the public-coin interactive protocol. The non-interactive variant in Section 4.2 derives both r and I from domain-separated hashes of the full transcript prefix. In the random oracle model, the same proof

applies with the standard Fiat-Shamir loss $\epsilon_{\text{FS}}(q_H)$, where q_H is the adversary's number of random-oracle queries. This term accounts for grinding over transcript prefixes and is negligible for polynomial q_H when the challenge and query hashes have λ -bit security.

Remark 2 (Knowledge Soundness). The theorem above is a statement-soundness result relative to certified public input commitments. A full *argument of knowledge* can be obtained by composing **FastMatMul** with an extractable input-commitment interface and a knowledge-sound SNARK. In that setting, the input-commitment extractor yields openings of $\text{cm}_A, \text{cm}_B, \text{cm}_C$ to row-wise encoded matrices, while the SNARK extractor yields the intermediate vectors and queried openings used in $\mathcal{R}_{\text{SNARK}}$. The same soundness argument then implies that any prover accepted with non-negligible probability is bound, except with the stated error, to matrices satisfying $\mathbf{AB} = \mathbf{C}$. We treat this as an orthogonal composition issue and focus the main theorem on the multiplication-checking protocol.

6. Implementation and Evaluation

We implement the multiplication-checking layer of **FastMatMul** and evaluate the current artifact, denoted **FastMatMul_{QA}**, against an optimized SNARK-in-circuit Freivalds baseline [1]. **FastMatMul_{QA}** uses multi-column Merkle openings, a Groth16 circuit proof, and a QA-NIZK linking layer for the opened encoded columns. All main experiments use a 256-bit prime field, rate $\rho = 1/2$, codeword length $n = 2k$, and $t = 128$ sampled positions. The experiments were run on a Linux server with an AMD EPYC 7B13 CPU (32 cores) and 254 GB RAM.

These measurements are layer-level measurements under the conditional linear-time code-commitment backend model of Section 4. They cover the implemented fold, opening, and linking terms of the multiplication-checking layer, but they do not independently implement or measure a standalone **CodeCom.Verify** component. Consequently, the empirical claims below should not be read as complete end-to-end deployment costs from raw inputs. Appendix C reports the full benchmark table, measurement coverage, parameter details, and dense-code encoding costs.

Primary matrix benchmark. Figure 2 shows the main scaling behavior. Freivalds constraints grow by approximately $4\times$ with each doubling of k , matching the expected $\mathcal{O}(k^2)$ scaling. In contrast, **FastMatMul_{QA}** grows by approximately $2\times$, matching the $\mathcal{O}(k)$ checking-layer scaling under the conditional linear-time backend model. The crossover in absolute constraint count occurs at $k = 256$. At $k = 1,024$, **FastMatMul_{QA}** uses 419,139 constraints versus 3,156,298 for Freivalds, a $7.53\times$ reduction. At $k = 4,096$, **FastMatMul_{QA}** uses 1,667,305 constraints versus 50,495,818 for Freivalds, a $30.29\times$ reduction.

The same benchmark shows a proving-time crossover at $k = 256$. At $k = 4,096$, **FastMatMul_{QA}** proves in 56.44 s versus 405.04 s for Freivalds, a $7.18\times$ speedup. **FastMatMul_{QA}** also completes $k = 8,192$ with 3,331,779

Term	Expression	Status in current artifact	Strict 128-bit deployment target
Freivalds randomization	$(k - 1)/ \mathbb{F} $	Uses a 256-bit prime field; for the largest reported row $k = 2^{13}$, this term is below 2^{-243} .	Choose $ \mathbb{F} $ so that the term fits the aggregate budget for the largest supported k .
Proximity sampling	$4(1 - \delta)^t$	Reported scaling rows use $t = 128$. If a backend certifies $\delta = 1/2$, this term is 2^{-126} .	Use $t = \lceil (\lambda + 2)/\log_2(1/(1 - \delta)) \rceil$ for proximity-only $2^{-\lambda}$; for a conservative per-term 2^{-131} budget at $\delta = 1/2$, use $t = 133$.
SNARK soundness	$\epsilon_{\text{SNARK}}(\lambda)$	Determined by the chosen SNARK backend; the CSV artifacts do not constitute a security certificate.	Instantiate the SNARK so this term is at most the allocated budget, e.g., 2^{-131} .
Codeword binding	$\epsilon_{\text{cc}}(\lambda)$	Modeled by Definition 4.1; standalone <code>CodeCom.Verify</code> is not independently implemented or measured.	Choose a concrete code-commitment backend proving global codeword binding and certified distance δ .
Opening binding	$\epsilon_{\text{bind}}(\lambda)$	Covered by Merkle/opening and QA-NIZK linking assumptions, union-bounded over all accepted openings.	Size hash/linking parameters so the union-bounded position-binding error is within the allocated budget.
Fiat-Shamir loss	$\epsilon_{\text{FS}}(q_H)$	Random-oracle-model loss for the non-interactive transcript.	Use domain-separated hashes and a query bound q_H such that the ROM loss remains within the aggregate budget.

TABLE 4. CONCRETE SOUNDNESS-BUDGET CHECKLIST FOR THEOREM 5.1. THE CURRENT EXPERIMENTS ARE SCALING MEASUREMENTS FOR THE CHECKING LAYER; A DEPLOYMENT CERTIFICATE MUST FILL IN THE BACKEND-SPECIFIC ENTRIES.

$\log k$	Constraints		Prove Time (s)		Verify Time		FastMatMul _{QA} Proof
	FastMatMul _{QA}	Freivalds	FastMatMul _{QA}	Freivalds	FastMatMul _{QA}	Freivalds	
7	55,059	49,610	0.78	0.58	0.10 s	1 ms	15,108 B
8	106,905	197,194	1.59	2.03	0.10 s	1 ms	29,028 B
9	210,985	788,810	3.35	6.89	0.10 s	1 ms	50,468 B
10	419,139	3,156,298	7.84	25.63	0.10 s	2 ms	66,468 B
11	835,065	12,622,154	21.16	102.07	0.10 s	2 ms	86,308 B
12	1,667,305	50,495,818	56.44	405.04	0.10 s	2 ms	104,388 B
13	3,331,779	—	178.48	—	0.10 s	—	125,508 B

TABLE 5. PRIMARY MATRIX-MULTIPLICATION BENCHMARK FOR FASTMATMUL_{QA} AND THE OPTIMIZED FREIVALDS SNARK BASELINE. FASTMATMUL_{QA} IS THE CURRENT IMPLEMENTATION USING MULTI-COLUMN MERKLE OPENINGS AND QA-NIZK LINKING. PROVING TIME EXCLUDES NATIVE MATRIX MULTIPLICATION AND SETUP; THESE ARE RECORDED SEPARATELY IN THE ARTIFACT. THE TABLE MEASURES THE MULTIPLICATION-CHECKING LAYER UNDER THE CONDITIONAL LINEAR-TIME CODE-COMMITMENT BACKEND MODEL AND DOES NOT INDEPENDENTLY MEASURE A STANDALONE CODECOM.VERIFY COMPONENT.

constraints and a 178.48 s proving time; Freivalds was not measured at that size in the current artifact.

Verification, proof size, and GPT-2 workload. FastMatMul_{QA} verification is nearly constant in the matrix benchmark at about 0.10 s, dominated by QA-NIZK link verification. Freivalds verification is smaller in absolute terms, between 1 and 2 ms, because it verifies a direct Groth16 proof without sampled-column linking material. The current artifact therefore trades slower verification for much smaller prover circuits and faster proving at medium and large dimensions. As Figure 3 shows, the FastMatMul_{QA} proof grows from 15,108 bytes at $k = 128$ to 104,388 bytes at $k = 4,096$ and 125,508 bytes at $k = 8,192$.

Figure 4 summarizes a GPT-2-style workload with sequence length 1,024, 36 matrix-multiplication claims, and 5 commitment groups. FastMatMul_{QA} reduces constraints from 76.8 million to 33.5 million (2.30 $\times$) and proving time from 408.69 s to 230.33 s (1.77 $\times$). Setup time is also lower (2,144.47 s versus 4,173.32 s). Verification remains slower for FastMatMul_{QA} (4.94 s versus 0.49 s), so latency-

sensitive deployments still need batching, amortization, or a more succinct opening/linking backend.

Summary. The experiments support the claimed asymptotic separation for the multiplication-checking layer and show concrete gains over the Freivalds SNARK baseline at large k . They do not close the remaining end-to-end backend gap: the standalone code-commitment backend relation `CodeCom.Verify` is still modeled rather than independently implemented and measured.

7. Related Work

Verifiable ML systems such as SafetyNets [4], zkCNN [12], vCNN [13], Mystique [2], and zkML [3] all encounter matrix multiplication as a dominant arithmetic cost. These systems typically verify matrix products either by arithmetizing the vector-matrix products directly or by using sumcheck/GKR-style machinery over layered arithmetic circuits. FastMatMul is complementary: it is a dedicated checking layer for the matrix multiplication

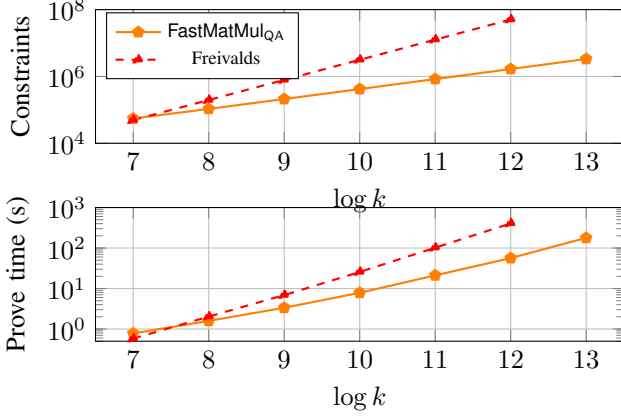


Figure 2. Primary matrix benchmark. Under the conditional linear-time backend model, FastMatMul_{QA} follows the expected $\mathcal{O}(k)$ checking-layer growth, whereas the Freivalds SNARK baseline follows $\mathcal{O}(k^2)$ growth.

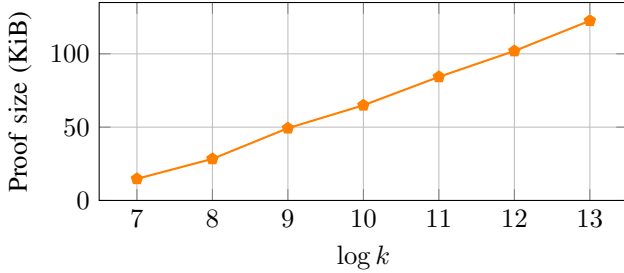


Figure 3. FastMatMul_{QA} proof size in the primary benchmark. The Groth16 proof component is constant-sized and the QA-NIZK link proof is 128 bytes; growth is dominated by sampled encoded columns and Merkle authentication material.

component, reducing the checked relation to $\mathcal{O}(tk + t \log n + E_{cc}(k))$ constraints under certified encoded commitments.

Dedicated matrix-verification protocols, including zk-Matrix [14], DualMatrix [15], zkVC [16], and the coding-theoretic line of Bennett et al. [17], explore different trade-offs among polynomial commitments, batching, prover work, and proof size. Our construction differs by exposing a SNARK-friendly local relation over committed codewords rather than committing to a particular polynomial-commitment backend. Relative to code-based proof systems such as Ligero [18], Brakedown [5], Orion [7], and FRI [19], the contribution is not a new general-purpose proof system; it is the problem-specific reduction that turns the matrix check into local codeword folds before arithmeticization. Appendix D gives a longer comparison table.

8. Conclusion

We presented FastMatMul, a matrix-multiplication checking layer whose SNARK relation has size $\mathcal{O}(tk + t \log n + E_{cc}(k))$ and becomes $\mathcal{O}(tk)$ with a linear-time code-commitment backend. The construction combines Freivalds’ randomized reduction with local proximity checks over linear error-correcting codes, and its security theorem

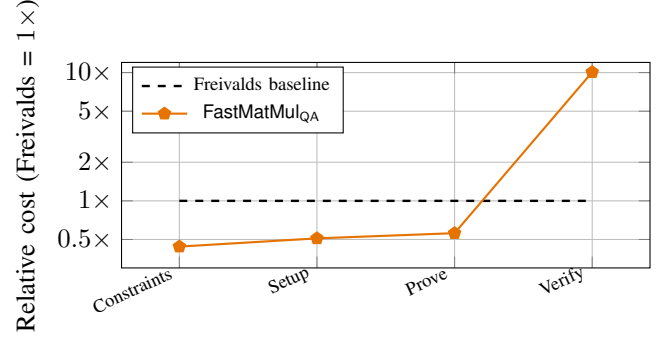


Figure 4. GPT-2 workload case study at sequence length 1,024 with 36 matrix-multiplication claims. Costs are normalized to the Freivalds baseline.

explicitly accounts for SNARK soundness, backend code-word binding, commitment binding, Freivalds failure, and proximity-sampling error.

Our implementation confirms the expected checking-layer separation against a Freivalds SNARK baseline: at $k = 4,096$, FastMatMul_{QA} gives a $30.29\times$ constraint reduction and a $7.18\times$ proving-time speedup; a GPT-2 workload case study gives a $2.30\times$ constraint reduction and a $1.77\times$ proving-time speedup. The main limitation remains explicit: the current artifact measures the fold/opening/linking layer under a conditional code-commitment backend model and does not yet independently implement or measure standalone CodeCom.Verify. Closing that backend gap, reducing verification overhead, and batching many matrix multiplications are the primary next steps toward a full deployment.

Ethics considerations

None.

LLM usage considerations

LLMs were used for editorial assistance in preparing this manuscript. All technical claims, proofs, experiments, and citations were checked by the authors, who are responsible for the content.

References

- [1] R. Freivalds, “Probabilistic machines can use less running time,” in *Information Processing 77 (Proceedings of IFIP Congress 77)*. Amsterdam, The Netherlands: North-Holland, 1977, pp. 839–842.
- [2] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang, “Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning,” in *30th USENIX Security Symposium (USENIX Security 21)*. Berkeley, CA, USA: USENIX Association, 2021, pp. 501–518.
- [3] B.-J. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang, “Zkml: An optimizing system for ml inference in zero-knowledge proofs,” in *Proceedings of the Nineteenth European Conference on Computer Systems*. New York, NY, USA: ACM, 2024, pp. 560–574.

- [4] Z. Ghodsi, T. Gu, and S. Garg, “Safetynets: Verifiable execution of deep neural networks on an untrusted cloud,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby, “Brakedown: Linear-time and field-agnostic snarks for r1cs,” in *Advances in Cryptology – CRYPTO 2023*. Cham: Springer, 2023, pp. 193–226.
- [6] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Advances in Cryptology – CRYPTO ’86*. Berlin, Heidelberg: Springer, 1987, pp. 186–194.
- [7] T. Xie, Y. Zhang, and D. Song, “Orion: Zero knowledge proof with linear prover time,” in *Advances in Cryptology – CRYPTO 2022*. Cham: Springer, 2022, pp. 299–328.
- [8] R. C. Merkle, “A certified digital signature,” in *Advances in Cryptology – CRYPTO ’89*. Berlin, Heidelberg: Springer, 1989, pp. 218–238.
- [9] J. T. Schwartz, “Fast probabilistic algorithms for verification of polynomial identities,” *Journal of the ACM*, vol. 27, no. 4, pp. 701–717, 1980.
- [10] R. Zippel, “Probabilistic algorithms for sparse polynomials,” in *Symbolic and Algebraic Computation (EUROSAM ’79)*. Berlin, Heidelberg: Springer, 1979, pp. 216–226.
- [11] T. P. Pedersen, “Non-interactive and information-theoretic secure verifiable secret sharing,” in *Advances in Cryptology – CRYPTO ’91*. Berlin, Heidelberg: Springer, 1991, pp. 129–140.
- [12] T. Liu, X. Xie, and Y. Zhang, “Zkcnns: Zero knowledge proofs for convolutional neural network predictions and accuracy,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2021, pp. 2968–2985.
- [13] S. Lee, H. Ko, J. Kim, and H. Oh, “vcnn: Verifiable convolutional neural network based on zk-snarks,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 4254–4270, 2024.
- [14] M. Cong, T. H. Yuen, and S.-M. Yiu, “zkmatrix: Batched short proof for committed matrix multiplication,” in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2024, pp. 289–305.
- [15] M. Cong, T. H. Yuen, and S. M. Yiu, “Dualmatrix: conquering zkcnns for large matrix multiplication,” *Cybersecurity*, vol. 9, no. 1, p. 126, 2026.
- [16] Y. Zhang, M. Zheng, X. Chen, J. Hu, W. Shi, L. Ju, Y. Solihin, and Q. Lou, “zkvc: Fast zero-knowledge proof for private and verifiable computing,” in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, IEEE. New York, NY, USA: IEEE, 2025, pp. 1–7.
- [17] H. Bennett, K. Gajulapalli, A. Golovnev, and E. Warton, “Matrix multiplication verification using coding theory,” *arXiv preprint arXiv:2309.16176*, 2023.
- [18] S. Ames, C. Hazay, Y. Ishai, and M. Venkatasubramanian, “Ligero: Lightweight sublinear arguments without a trusted setup,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2017, pp. 2087–2104.
- [19] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast reed-solomon interactive oracle proofs of proximity,” in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 14:1–14:17.
- [20] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, “Delegating computation: Interactive proofs for muggles,” *Journal of the ACM*, vol. 62, no. 4, pp. 1–64, 2015.

Appendix A.

Detailed Proof of Theorem 5.1

Proof. We establish completeness and soundness for the interactive protocol, and then explain the Fiat–Shamir variant.

Perfect completeness. Suppose the prover is honest and $\mathbf{AB} = \mathbf{C}$. Let $\mathbf{D} := \mathbf{AB} - \mathbf{C} = \mathbf{0}$. For any challenge vector $\mathbf{r}$, we have $\mathbf{rD} = \mathbf{0}$, so

$$\mathbf{y} = \mathbf{xB} = (\mathbf{rA})\mathbf{B} = \mathbf{r}(\mathbf{AB}) = \mathbf{rC} = \mathbf{z}.$$

For each intermediate vector $v \in \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$, the honest prover commits to $\text{Enc}(v)$ and supplies backend proof material σ_v accepted by `CodeCom.Verify`. For any queried index $j \in [n]$, linearity of $\mathcal{C}$ gives

$$\text{Fold}(\text{Enc}(\mathbf{A})^{(j)}, \mathbf{r}) = \text{Enc}(\mathbf{rA})_j = \text{Enc}(\mathbf{x})_j,$$

and analogously for the $\mathbf{B}$ and $\mathbf{C}$ folds. Since $\mathbf{y} = \mathbf{z}$, the equality check between their encoded openings also holds at every coordinate. All authenticated openings and backend checks are honest, so the verifier accepts with probability one.

Bad-event decomposition. Let Acc be the event that the verifier accepts a false committed matrix statement, and assume $\mathbf{AB} \neq \mathbf{C}$. Define $\text{Bad}_{\text{SNARK}}$ as acceptance of a false $\mathcal{R}_{\text{SNARK}}$ instance, Bad_{cc} as a failure of the global codeword-binding property for an accepted intermediate root, and Bad_{bind} as two inconsistent accepted openings for the same root and position. By the assumed backend properties,

$$\Pr[\text{Bad}_{\text{SNARK}}] \leq \epsilon_{\text{SNARK}}(\lambda),$$

$$\Pr[\text{Bad}_{\text{cc}}] \leq \epsilon_{\text{cc}}(\lambda),$$

$$\Pr[\text{Bad}_{\text{bind}}] \leq \epsilon_{\text{bind}}(\lambda).$$

Conditioned on the complement of these events, every accepting transcript has a valid SNARK witness, every accepted intermediate root is globally bound to the full codeword $\text{Enc}(v)$ for its witness vector v , and every accepted opening value is uniquely determined by its root and queried coordinate. The certified input roots are fixed before the Freivalds challenge is sampled, and the intermediate roots are fixed before the sampled query tuple is sampled.

Freivalds randomization. Under the same conditioning, the matrices bound to $\text{cm}_A, \text{cm}_B, \text{cm}_C$ are fixed and satisfy $\mathbf{AB} \neq \mathbf{C}$. Let $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$. There exists a column $\mathbf{d}_j$ of $\mathbf{D}$ with $\mathbf{d}_j \neq \mathbf{0}$. The condition $\mathbf{rD} = \mathbf{0}$ implies

$$P_j(r) := \langle \mathbf{r}, \mathbf{d}_j \rangle = \sum_{i=0}^{k-1} (\mathbf{D})_{i,j} r^i = 0.$$

Since P_j is a nonzero polynomial of degree at most $k-1$ and r is sampled after the input commitments are fixed, Lemma 3.2 gives

$$\Pr_{r \leftarrow \mathbb{F}^k}[\mathbf{rD} = \mathbf{0}] \leq \Pr_{r \leftarrow \mathbb{F}^k}[P_j(r) = 0] \leq \frac{k-1}{|\mathbb{F}|}.$$

Proximity sampling. Now condition further on $\mathbf{rD} \neq \mathbf{0}$. No vectors can simultaneously satisfy all four global relations:

$$\mathbf{x} = \mathbf{rA}, \quad \mathbf{y} = \mathbf{xB}, \quad \mathbf{z} = \mathbf{rC}, \quad \mathbf{y} = \mathbf{z}.$$

Otherwise the four equalities would imply $\mathbf{rAB} = \mathbf{rC}$, contradicting $\mathbf{rD} \neq \mathbf{0}$. Thus at least one relation is false.

Consider $\mathbf{x} \neq \mathbf{r}\mathbf{A}$; the other cases are identical. Injectivity and distance of the code imply that the two codewords $\text{Enc}(\mathbf{x})$ and $\text{Enc}(\mathbf{r}\mathbf{A})$ disagree on at least δn coordinates. By code linearity, the sampled fold value from the committed rows of $\mathbf{A}$ equals $\text{Enc}(\mathbf{r}\mathbf{A})_j$ at coordinate j . Since accepted openings are bound to the committed codeword $\text{Enc}(\mathbf{x})$, the fold check rejects whenever j is in the disagreement set. A single uniformly sampled coordinate misses this disagreement set with probability at most $1 - \delta$, and t independent sampled coordinates all miss it with probability at most $(1 - \delta)^t$. Union-bounding over the four possible violated relations yields $4(1 - \delta)^t$.

Combining the bounds and parameters. Summing the backend, Freivalds, and proximity errors gives

$$\begin{aligned} \Pr[\text{Acc} \mid \mathbf{AB} \neq \mathbf{C}] &\leq \epsilon_{\text{SNARK}}(\lambda) + \epsilon_{\text{cc}}(\lambda) + \epsilon_{\text{bind}}(\lambda) \\ &\quad + \frac{k-1}{|\mathbb{F}|} + 4(1 - \delta)^t. \end{aligned}$$

For $\lambda = 128$, a 256-bit field makes $(k-1)/|\mathbb{F}|$ negligible for the reported dimensions. The proximity term is at most $2^{-\lambda}$ when $t = \lceil (\lambda + 2)/\log_2(1/(1 - \delta)) \rceil$; for example, if the backend certifies $\delta = 1/2$, then $t = 130$ suffices for the proximity term alone. For a conservative aggregate 128-bit budget over all terms, one should allocate the budget across terms, e.g., by targeting each term at 2^{-131} .

Fiat–Shamir variant. The non-interactive transcript derives both r and I from domain-separated hashes of the transcript prefix, including the public input roots and intermediate roots. In the random oracle model, the preceding public-coin proof applies with the standard Fiat–Shamir loss $\epsilon_{\text{FS}}(q_H)$ for an adversary that makes q_H random-oracle queries. This proves Theorem 5.1. $\square$

Appendix B.

Probabilistic Verification of Reed–Solomon Codewords

Let $\mathbb{F}$ be a finite field, let $\mathbf{m} = (a_0, \dots, a_{k-1}) \in \mathbb{F}^k$, and let $\mathbf{c} = (c_0, \dots, c_{n-1}) \in \mathbb{F}^n$ for $n \geq k$. Let $S = \{w^0, w^1, \dots, w^{n-1}\} \subseteq \mathbb{F}$ be the evaluation domain. Define the message polynomial $f(X) = \sum_{i=0}^{k-1} a_i X^i$ and let $g(X)$ be the unique polynomial of degree less than n satisfying $g(w^i) = c_i$ for every i . The goal is to test whether $f(X) \equiv g(X)$, i.e., whether $\mathbf{c}$ is the Reed–Solomon encoding of $\mathbf{m}$.

The verifier samples $z \leftarrow_{\$} \mathbb{F} \setminus S$, computes $v_1 = f(z)$ by Horner evaluation, and computes $v_2 = g(z)$ by barycentric interpolation:

$$v_2 = \frac{\sum_{i=0}^{n-1} \frac{\lambda_i}{z-w^i} c_i}{\sum_{i=0}^{n-1} \frac{\lambda_i}{z-w^i}},$$

where λ_i are the barycentric weights for S . The verifier accepts iff $v_1 = v_2$.

If $f(X) \not\equiv g(X)$, then $h(X) = f(X) - g(X)$ is a nonzero polynomial of degree less than n . By Schwartz–Zippel,

$$\Pr_{z \leftarrow_{\$} \mathbb{F}} [f(z) = g(z)] \leq \frac{n-1}{|\mathbb{F}|},$$

which is negligible when $|\mathbb{F}|$ is sufficiently large.

Appendix C. Additional Evaluation Details

Parameter	Value in main experiments
Field size	256-bit prime field
Server	AMD EPYC 7B13, 32 cores, 254 GB RAM, Linux
Matrix dimensions	FastMatMul _{QA} : 2^7-2^{13} ; Freivalds: 2^7-2^{12}
Code rate	$\rho = 1/2$
Codeword length	$n = 2k$
Sampled positions	$t = 128$
Primary linking layer	QA-NIZK with multi-column Merkle openings
Backend model	Conditional linear-time CodeCom.Verify backend
Strict 128-bit proximity target	$t = \lceil (\lambda + 2)/\log_2(1/(1 - \delta)) \rceil$; $t = 130$ when $\delta = 1/2$

TABLE 6. CONCRETE PARAMETERS USED IN THE MAIN EXPERIMENTS.

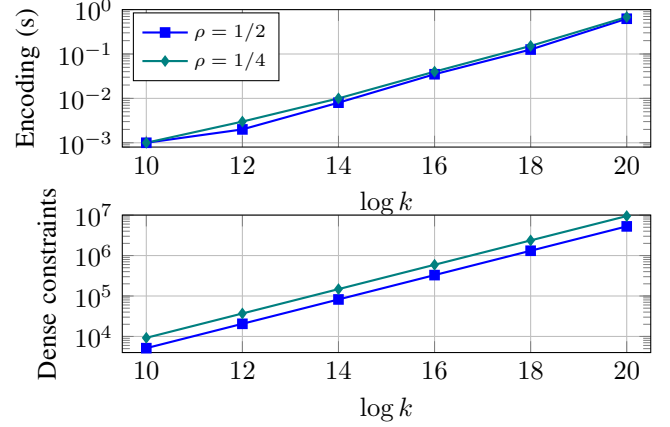


Figure 5. Encoding time and dense codeword-validity constraints for general constant-rate codes. Encoding stays below one second up to $\log k = 20$, while dense backend checks grow quadratically.

Metric	Covered?	Interpretation
Constraint count	Yes	Checking-layer circuit size under the conditional backend model.
Online proving time	Yes	Excludes native matrix multiplication and one-time setup.
Verification time	Yes	Includes current QA-NIZK/opening verification overhead.
Proof size	Yes	Includes Groth16, sampled openings, and QA-NIZK material.
Setup time	Partial	Recorded for the GPT-2 workload; setup/key sizes are not reported.
Standalone CodeCom.Verify	No	Required to close the end-to-end backend gap.
Peak memory and variance	No	Needed for a full systems artifact evaluation.

TABLE 7. MEASUREMENT COVERAGE IN THE CURRENT ARTIFACT.

Appendix D. Extended Related Work

Approach	Main work	checked	Relation to FastMatMul
Freivalds SNARK	in Vector-matrix products inside the circuit		Baseline measured in this paper; circuit cost is $\mathcal{O}(k^2)$.
Sumcheck/GKR systems	Multilinear-extension evaluations and rounds		Powerful for layered arithmetic circuits; usually needs a specialized verifier or compiler.
Dedicated matrix VC	Polynomial commitments or inner-product arguments	com-	Often gives short proofs; assumptions and batching model differ from our SNARK-checking layer.
Code-based SNARKs	General proving committed codewords	circuit over	Supplies the committed-codeword viewpoint; naïvely proving Freivalds still leaves an $\mathcal{O}(k^2)$ circuit.
FastMatMul	Local codeword folds plus backend binding		Reduces the checked matrix relation to $\mathcal{O}(tk + t \log n + E_{cc}(k))$ constraints.

TABLE 8. QUALITATIVE POSITIONING AGAINST RELATED MATRIX-VERIFICATION APPROACHES.

The sumcheck protocol [20] is a powerful interactive proof technique for verifying evaluations of multivariate polynomials. SafetyNets [4] and zkCNN [12] use this approach for neural-network verification. Unlike sumcheck-based methods, our proximity-based checks compare codeword coordinates through inner products rather than evaluating polynomial extensions. The resulting relation is directly expressible as R1CS or PlonK constraints, while sumcheck-based systems typically require a specialized verifier or a dedicated GKR-to-SNARK compilation step.